

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería
Ingeniería en Sistema de la Información y Ciencias de la Computación
Algoritmos
Ing. Moises Eduardo Velasquez Oliva



Practica 1

José Carlos Rodríguez Morales
Carné: 0910-26-14369

15 de Septiembre de 2026

Introducción

En esta práctica se desarrollará una aplicación llamada “HojaClara”, pensada para facilitar el trabajo de un pequeño despacho de contabilidad. La idea surge debido a la necesidad de contar con una herramienta sencilla que permita organizar datos, realizar cálculos y actualizar los resultados de manera automática, sin depender de una conexión a internet o de programas que requieran una licencia.

A lo largo de la práctica se irá construyendo la aplicación paso a paso, comenzando con una cuadrícula similar a una hoja de cálculo y agregando diferentes funciones que permitan ingresar datos, realizar operaciones y trabajar con fórmulas. También se busca que la aplicación sea fácil de utilizar y que pueda adaptarse a las necesidades de las personas que la utilicen.

En este documento se presenta de forma resumida la problemática y mi solución. También se adjuntan el Pseudocódigo y diagramas de flujo realizados en el programa de Pseint.

Resumen del problema principal:

La Licenciada Silva Xitumul necesita un programa de hoja de cálculo en el cual pueda solucionar los siguientes problemas como:

Hacer todo a mano (El recálculo manual): Cada vez que un cliente entrega un recibo tarde o hay que corregir un dato, Silvia tiene que rastrear en sus hojas todo lo que dependía de ese número y volver a calcularlo usando una calculadora. Por ejemplo, en una planilla, modificar el salario base de una persona obliga a recalcular bonos, descuentos del IGSS y el ISR retenido, Corregir una planilla entera por culpa de un solo cambio le quita en promedio cuarenta minutos de su tiempo.

Los errores en cadena (Errores de arrastre): Como los cálculos se hacen a mano o reciclando hojas impresas de meses anteriores, es muy común que a alguien en la oficina se le olvide actualizar una celda que depende de otra. Recientemente, por un descuido de este tipo, se calculo mal el total de una cotización y Silvia tuvo que pagar la diferencia de su bolsa para no quedar mal con el cliente.

El problema del internet (Dependencia de internet): Las herramientas en la nube que han probado dejan de servir cuando se cae el internet. Esto les pasa con mucha frecuencia, sobre todo en los últimos días del mes, que es justo cuando tienen más carga de trabajo por los cierres contables. Por lo tanto, urgen de un sistema que funcione siempre, sin depender de que la conexión sea estable.

Necesidad de formatos a la medida (Falta de control sobre el formato): Cada cliente tiene gustos diferentes algunos exigen ver los montos en quetzales con dos decimales, otros quieren que los saldos negativos destaquen con otro color, o necesitan sumar o promediar de golpe todos los gastos de una semana. Las opciones gratuitas que conocen no dejan hacer esto fácilmente, y comprar licencias de software especializado se sale completamente del presupuesto del despacho.

En resumen, lo que realmente necesita: Silvia no pide una aplicación excesivamente bonita o compleja. Ella solo requiere algo directo que funcione en las tres computadoras del despacho sin tener que instalar programas raros. Quiere una cuadrícula parecida a Excel donde pueda escribir datos y fórmulas, que actualice las cuentas por sí sola al cambiar un número, y que le permita ir agregando detalles según lo que le pidan sus clientes.

Mi propuesta

Para solucionar esto mi propuesta es mi proyecto el cual esta realizado en javascript el cual esta dividido en los siguientes puntos:

Para la seguridad y la lógica: En lugar de usar atajos, creamos un sistema que lee la fórmula pedacito por pedacito y la resuelve paso a paso usando pilas. Esto hace que la página sea segura y demuestra que de verdad aplicamos lo que aprendimos sobre algoritmos y estructuras de datos.

Para evitar que se trabe (Estabilidad): Hicimos un algoritmo que lleva un historial de las celdas para detectar si hay dependencias infinitas. Si el usuario hace una referencia circular por error, en vez de que la pestaña se congele, el sistema detiene el cálculo y muestra el mensaje "#CIRCULAR!". Así, la página sigue funcionando con normalidad.

Para que sea fácil y rápido de usar: Le agregamos opciones para arrastrar fórmulas (que se actualizan solas), usar funciones como SUMA o PROMEDIO, y el clásico Deshacer/Rehacer (Ctrl+Z). Una hoja de cálculo donde haces todo a mano no sirve de mucho; con esto le ahorramos trabajo al usuario y le perdonamos los errores, haciendo que se sienta como una herramienta real y profesional.

Para no perder datos: Le pusimos guardado automático en el navegador (localStorage) y una opción para exportar a CSV. Así, si recargas la página por accidente, no pierdes nada. Además, puedes descargar tu trabajo y abrirlo en Excel, dándole una utilidad real al proyecto

Pseudocódigo y diagramas de flujo

Generacion de la cuadrícula

Algoritmo Generacion_Cuadrícula

Definir fila, columna Como Entero

Definir lineaFila, encabezado Como Cadena

Escribir "--- INICIANDO RENDERIZADO DE LA HOJA ---"

encabezado <- " "

Para columna <- 0 Hasta 4 Hacer

Segun columna Hacer

0: encabezado <- encabezado + " A "

1: encabezado <- encabezado + " B "

2: encabezado <- encabezado + " C "

3: encabezado <- encabezado + " D "

4: encabezado <- encabezado + " E "

FinSegun

FinPara

Escribir encabezado

Escribir " -----"

Para fila <- 0 Hasta 9 Hacer

Si fila < 9 Entonces

lineaFila <- ConvertirATexto(fila + 1) + " | "

Sino

lineaFila <- ConvertirATexto(fila + 1) + " | "

FinSi

Para columna <- 0 Hasta 4 Hacer

lineaFila <- lineaFila + "[] "

FinPara

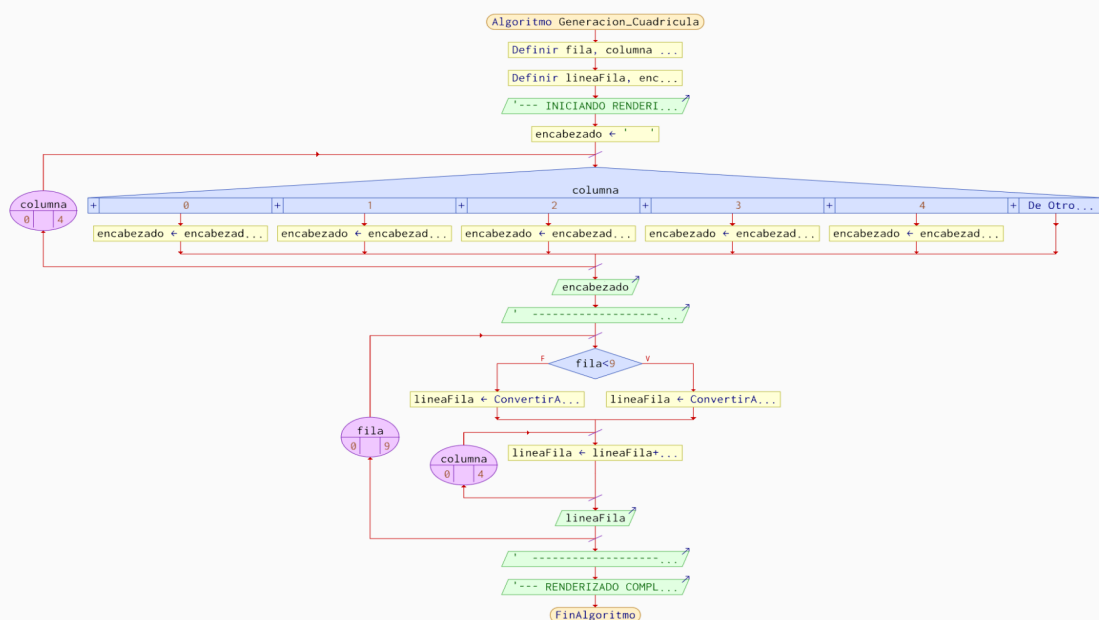
Escribir lineaFila

FinPara

Escribir " -----"

Escribir "--- RENDERIZADO COMPLETADO ---"

FinAlgoritmo



Tokenización de una fórmula

Algoritmo Tokenizador_Formula

Definir formula, tokenActual, caracter Como Cadena

Definir i Como Entero

formula = "A1+B2*3"

tokenActual = ""

// Recorrer la cadena caracter por caracter

Para i = 1 Hasta Longitud(formula) Hacer

caracter = Subcadena(formula, i, i)

Si caracter = "+" O caracter = "-" O caracter = "*" O caracter = "/" O caracter = "(" O caracter = ")" Entonces

Si tokenActual <> "" Entonces

Escribir "Guardar Token: ", tokenActual

tokenActual = ""

FinSi

Escribir "Guardar Operador: ", caracter

Sino

tokenActual = Concatenar(tokenActual, caracter)

FinSi

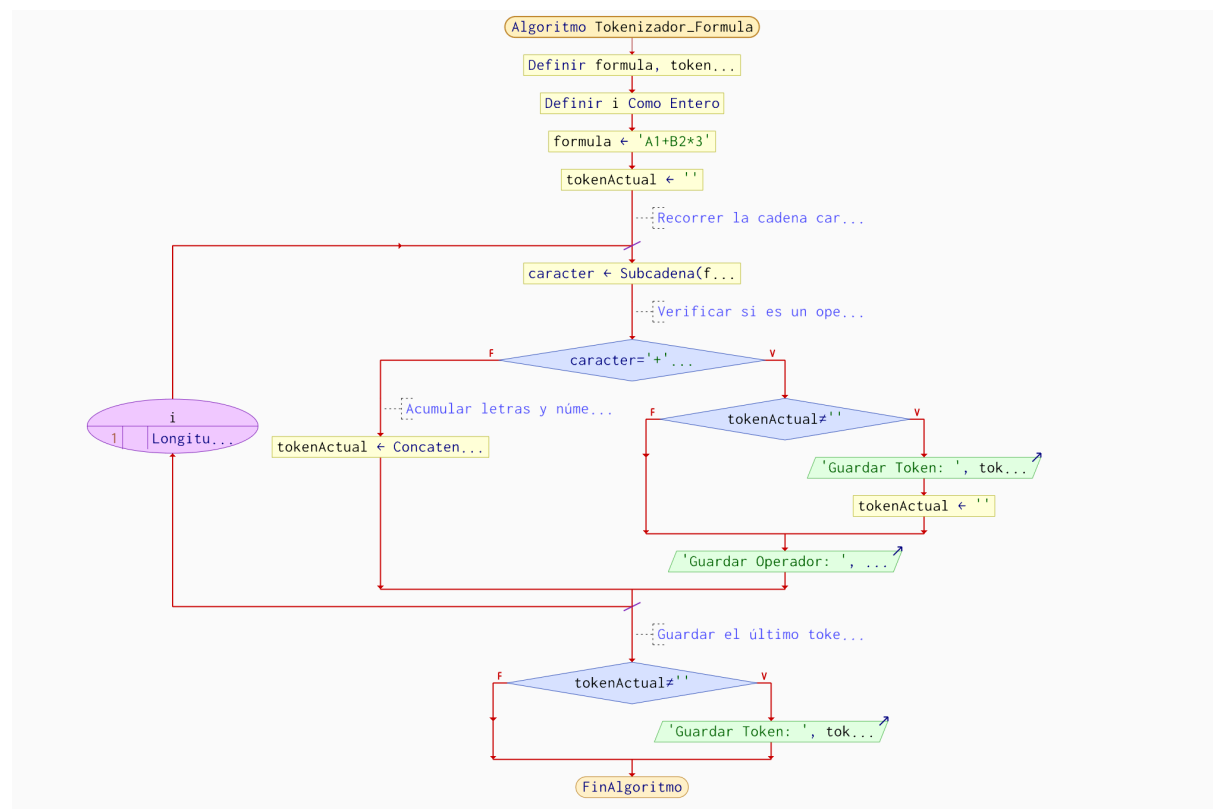
FinPara

Si tokenActual <> "" Entonces

Escribir "Guardar Token: ", tokenActual

FinSi

FinAlgoritmo



Evaluacion de la expresion tokenizada

Algoritmo Evaluacion_Expresion

Definir tokenActual Como Cadena

Definir topeValores, topeOps Como Entero

Dimension pilaValores(10)

Dimension pilaOps(10)

topeValores = 0

topeOps = 0

tokenActual = "+"

Si tokenActual = "Numero_o_Celda" Entonces

topeValores = topeValores + 1

pilaValores(topeValores) = 5 // Se empuja a la pila de valores

Sino

Si tokenActual = "+" O tokenActual = "*" Entonces

Mientras topeOps > 0 Y pilaOps(topeOps) = "*" Hacer

Escribir "Sacar 2 valores, multiplicar y guardar resultado"

topeOps = topeOps - 1

FinMientras

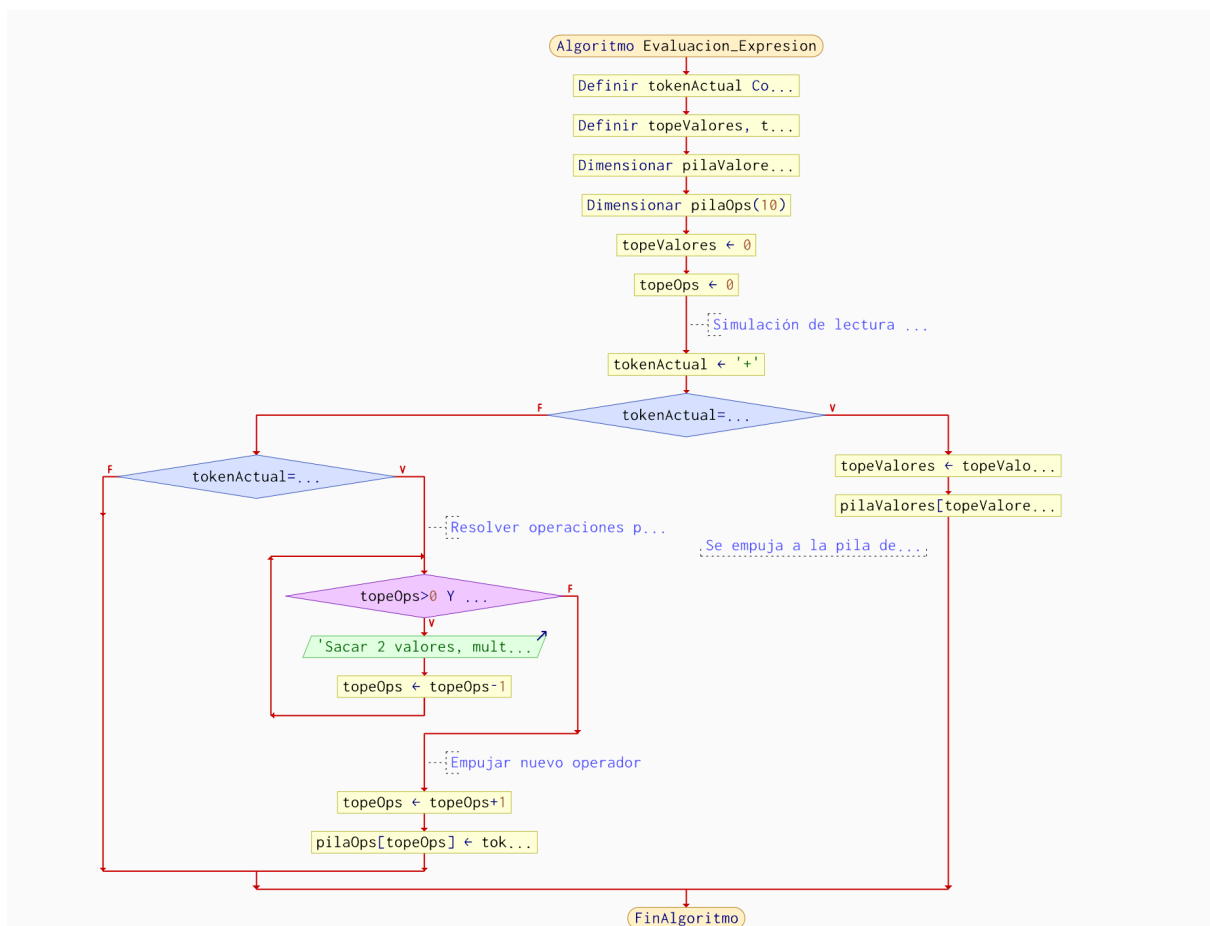
topeOps = topeOps + 1

pilaOps(topeOps) = tokenActual

FinSi

FinSi

FinAlgoritmo



Recalculo en cadena:

```
Funcion respuesta <- DetectarCiclo(column, fila)
  Definir respuesta Como Logico
  respuesta = Falso
FinFuncion
```

Algoritmo Recalculo_Matriz

```
Definir fila, col Como Entero
Definir valorIngresado Como Cadena
```

```
Para col = 0 Hasta 4 Hacer
  Para fila = 0 Hasta 9 Hacer
```

```
    valorIngresado = "=A1+5" // Simulación de una celda
```

```
    Si DetectarCiclo(col, fila) = Verdadero Entonces
      Escribir "Asignar valor: #CIRCULAR!"
```

```
    Sino
```

```
      Si Subcadena(valorIngresado, 1, 1) = "=" Entonces
        Escribir "Tokenizar y Evaluar Formula"
```

```
      Sino
```

```
        Escribir "Copiar valor estatico a computedValue"
```

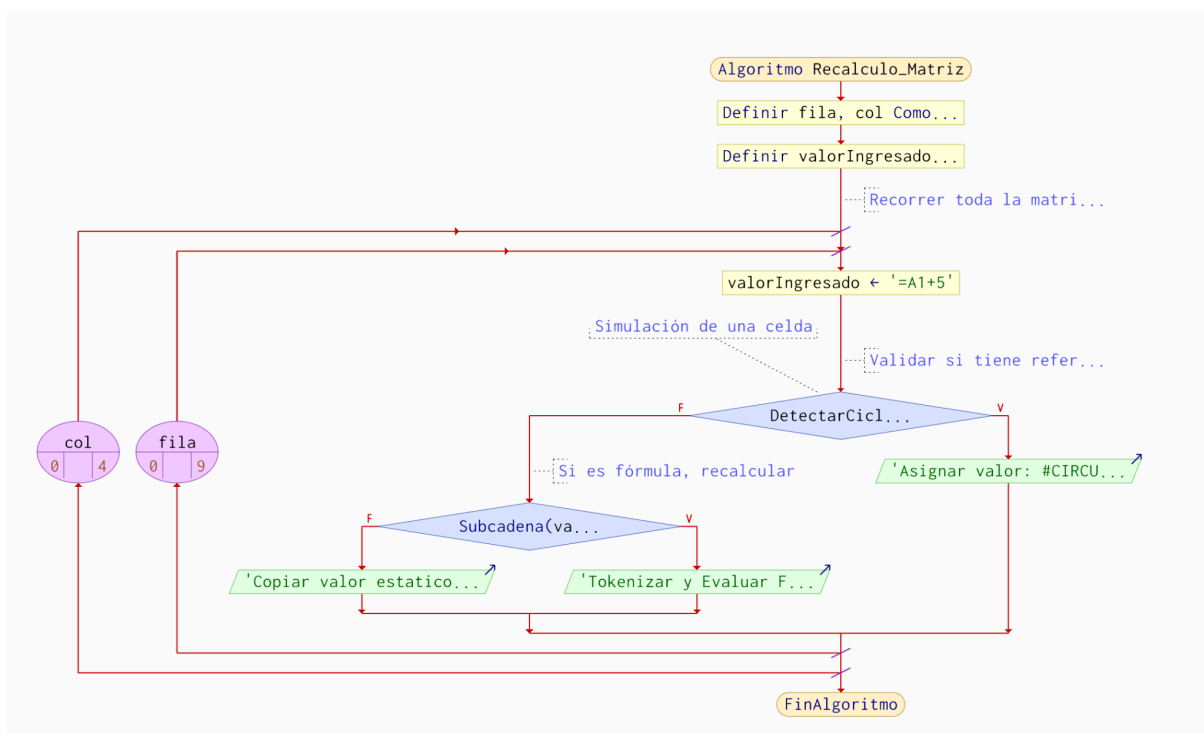
```
      FinSi
```

```
    FinSi
```

```
  FinPara
```

```
FinPara
```

```
FinAlgoritmo
```



Deteccion de referencias circulares:

Algoritmo Deteccion_Circular_Rekursiva

Definir hayCircular Como Logico

Definir rutaVisitados Como Cadena

rutaVisitados = ""

hayCircular = RevisarDependencias("A1", rutaVisitados)

Si hayCircular = Verdadero Entonces

Escribir "Error: Bucle Infinito Detenido"

Sino

Escribir "Formula Valida"

FinSi

FinAlgoritmo

Funcion ciclo = RevisarDependencias(celdaActual, visitados)

Definir ciclo Como Logico

Definir dependenciaHija Como Cadena

ciclo = Falso

Si celdaActual = visitados Entonces

ciclo = Verdadero

Sino

visitados = Concatenar(visitados, celdaActual)

dependenciaHija = "B1" // Simulacion de extraer una celda de la formula

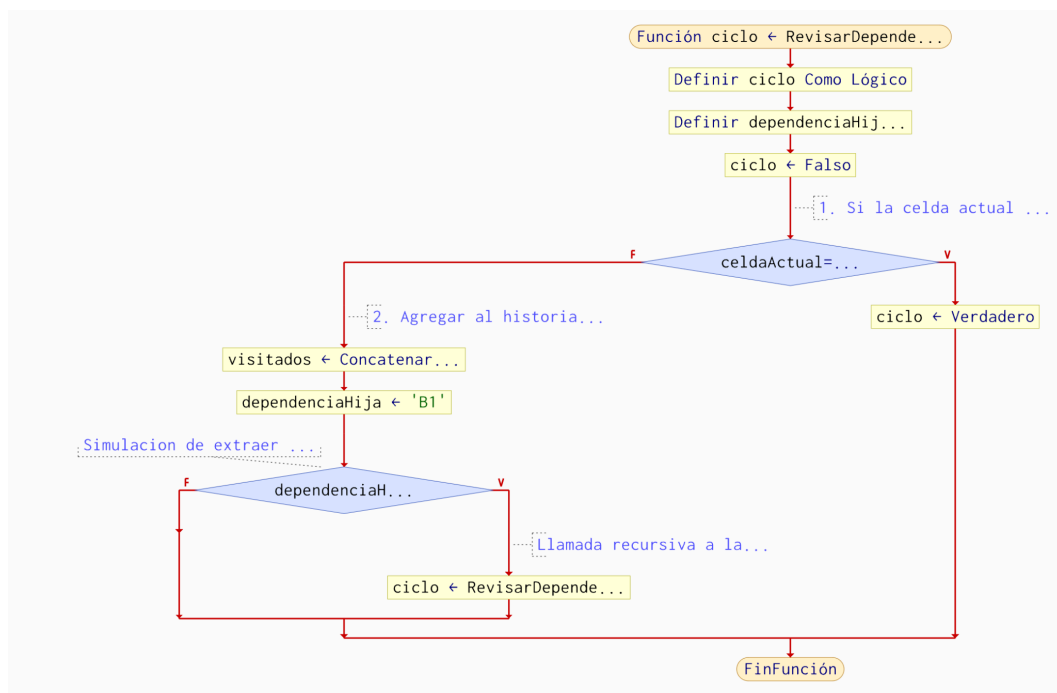
Si dependenciaHija <> "" Entonces

ciclo = RevisarDependencias(dependenciaHija, visitados)

FinSi

FinSi

FinFuncion



Diseño de la estructura de datos

La hoja de cálculo HojaClara representa su información en memoria mediante un modelo matricial bidimensional apoyado por un objeto que separa la entrada bruta del usuario de la salida evaluada.

Representación Interna del Estado (`STATE`)

El estado global de la aplicación se estructura como un arreglo de arreglos de objetos (Matriz Bidimensional), donde la primera dimensión corresponde a las columnas (0 a 4, mapeadas alfabéticamente de la `A` a la `E`) y la segunda dimensión corresponde a las filas (0 a 9, mapeadas numéricamente del `1` al `10`).

Propiedad	Tipo de Dato	Descripción	Ejemplo de Entrada
value	String	Contenido en crudo ingresado por el usuario (texto plano, valor numérico o fórmula).	"=A1+10"
computedValue	String / Number	Resultado procesado y renderizado tras la evaluación del motor algorítmico.	25

Representación y Resolución de Dependencias

Las dependencias entre celdas no se almacenan como enlaces estáticos, sino que se evalúan dinámicamente como un **Grafo Dirigido Implícito** durante la fase de recálculo:

- Extracción de Nodos:** Al evaluar el contenido de una celda, el tokenizador analiza la cadena `value` mediante expresiones regulares (`/[A-Z]+\d+/g`) para identificar las celdas de las cuales depende (nodos origen).
- Relación Origen - Destino:** La celda actual consulta la propiedad `computedValue` de cada celda referenciada dentro de la matriz `STATE`.
- Detección de Ciclos (Control de Recursión):** Para evitar bucles infinitos en el grafo de dependencias (por ejemplo, A1 depende de B1 y B1 depende de A1), la función recursiva `hasCircularReference` utiliza un conjunto **Set** que registra la ruta de navegación activa. Si un nodo intenta visitarse a sí mismo en la misma secuencia de llamadas, la evaluación se detiene y asigna el estado `#CIRCULAR!`.